

```
In [1]: import numpy as np # liner algebra
import pandas as pd # dtata processing, csv file I/O (e.g. pd. read_csv)
```

```
In [24]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
#plt.style.use('fivethirtyeight')
import warnings
warnings.filterwarnings('ignore')
```

```
In [3]: iris=pd.read_csv(r'D:\AI ML\FSDS AI\FSDS AI CLASS DOCS AND CODE\November\28th Nov -
```

```
In [4]: iris
```

```
Out[4]:
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa
...	...	...	...	...	...	...
145	146	6.7	3.0	5.2	2.3	Iris-virginica
146	147	6.3	2.5	5.0	1.9	Iris-virginica
147	148	6.5	3.0	5.2	2.0	Iris-virginica
148	149	6.2	3.4	5.4	2.3	Iris-virginica
149	150	5.9	3.0	5.1	1.8	Iris-virginica

150 rows × 6 columns

```
In [5]: iris.head()
```

```
Out[5]:
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

```
In [6]: iris.drop('Id',axis=1,inplace=True)
```

```
In [7]: iris.head()
```

```
Out[7]:
```

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

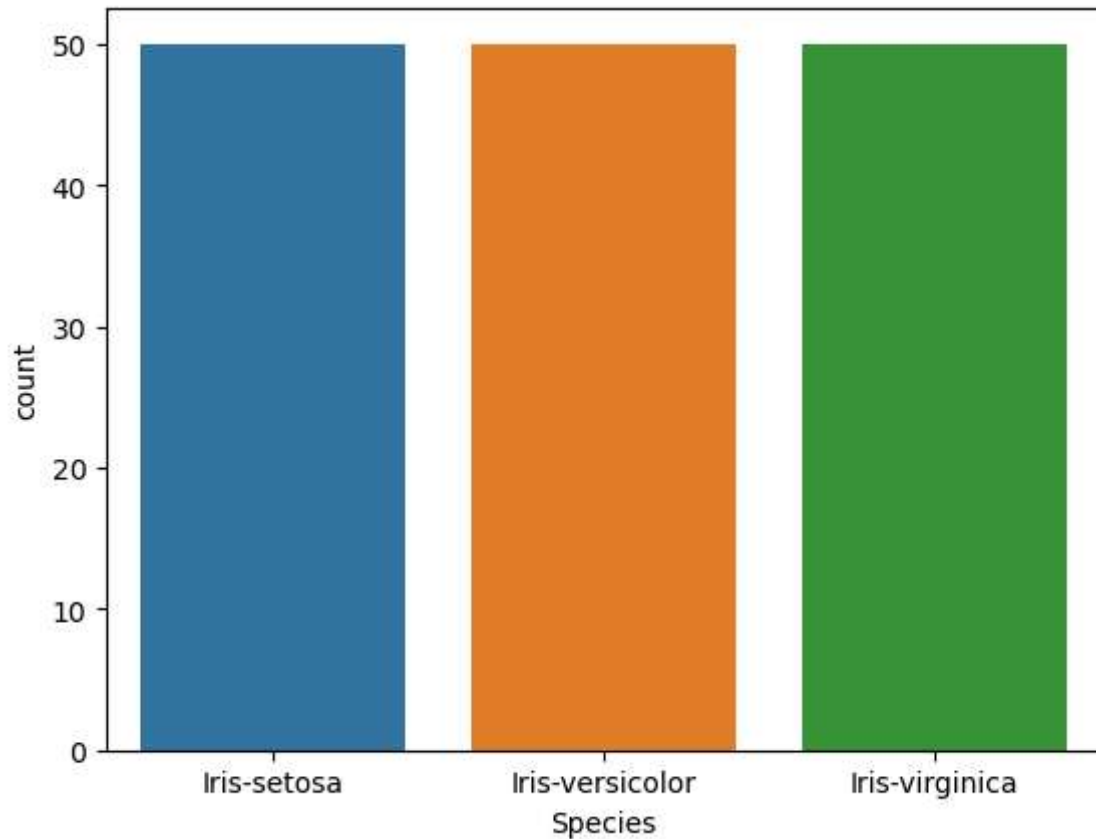
```
In [8]: iris.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 150 entries, 0 to 149  
Data columns (total 5 columns):  
#   Column          Non-Null Count  Dtype  
---  -  
0   SepalLengthCm    150 non-null    float64  
1   SepalWidthCm     150 non-null    float64  
2   PetalLengthCm    150 non-null    float64  
3   PetalWidthCm     150 non-null    float64  
4   Species          150 non-null    object  
dtypes: float64(4), object(1)  
memory usage: 6.0+ KB
```

```
In [28]: iris['Species'].value_counts()
```

```
Out[28]: Species  
Iris-setosa      50  
Iris-versicolor  50  
Iris-virginica   50  
Name: count, dtype: int64
```

```
In [9]: sns.countplot(x='Species', data=iris, hue='Species')  
plt.show()
```



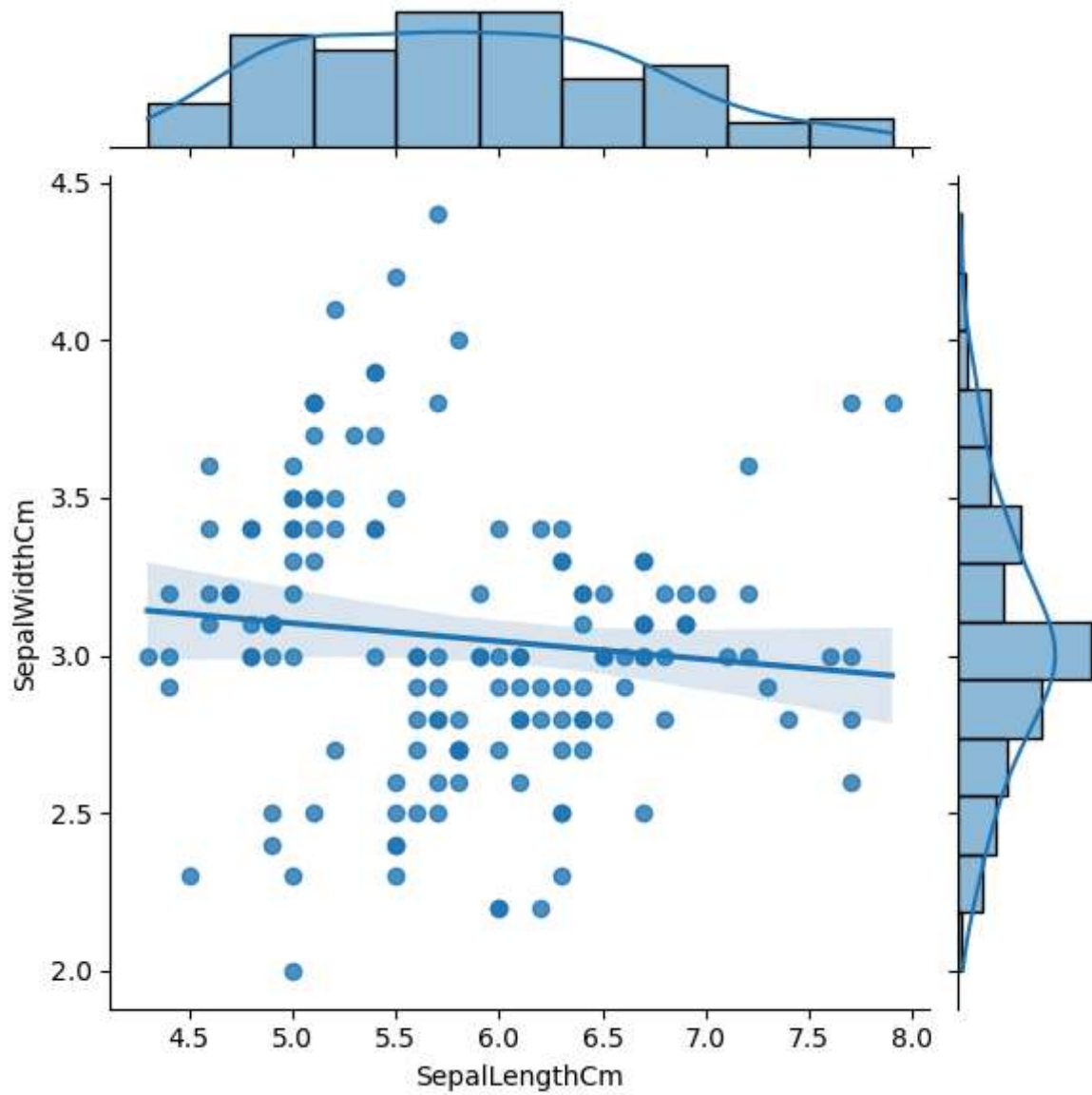
```
In [36]: iris.head()
```

```
Out[36]:
```

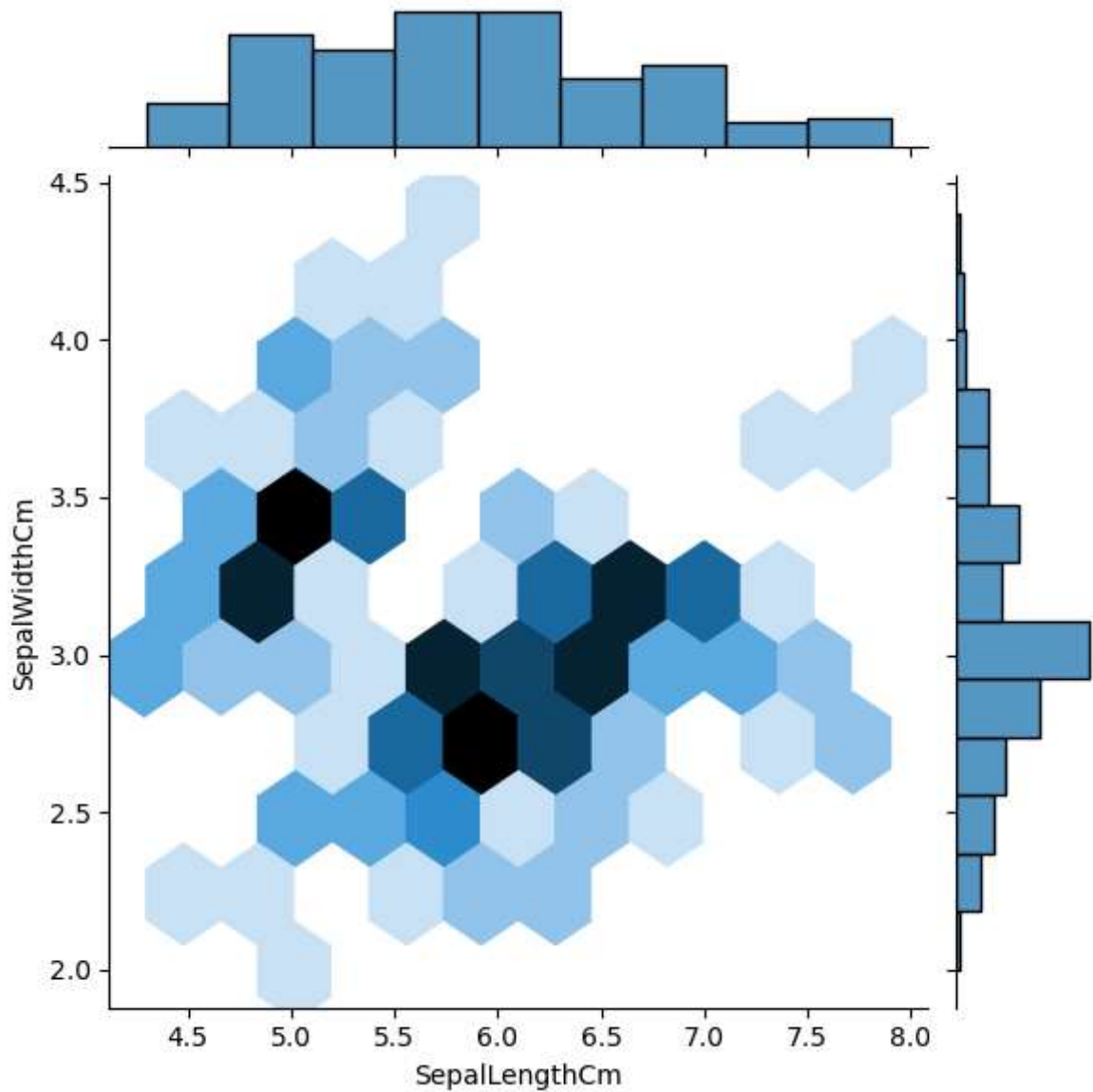
	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

```
In [16]: fig=sns.jointplot(x='SepalLengthCm',y='SepalWidthCm',data=iris)
```

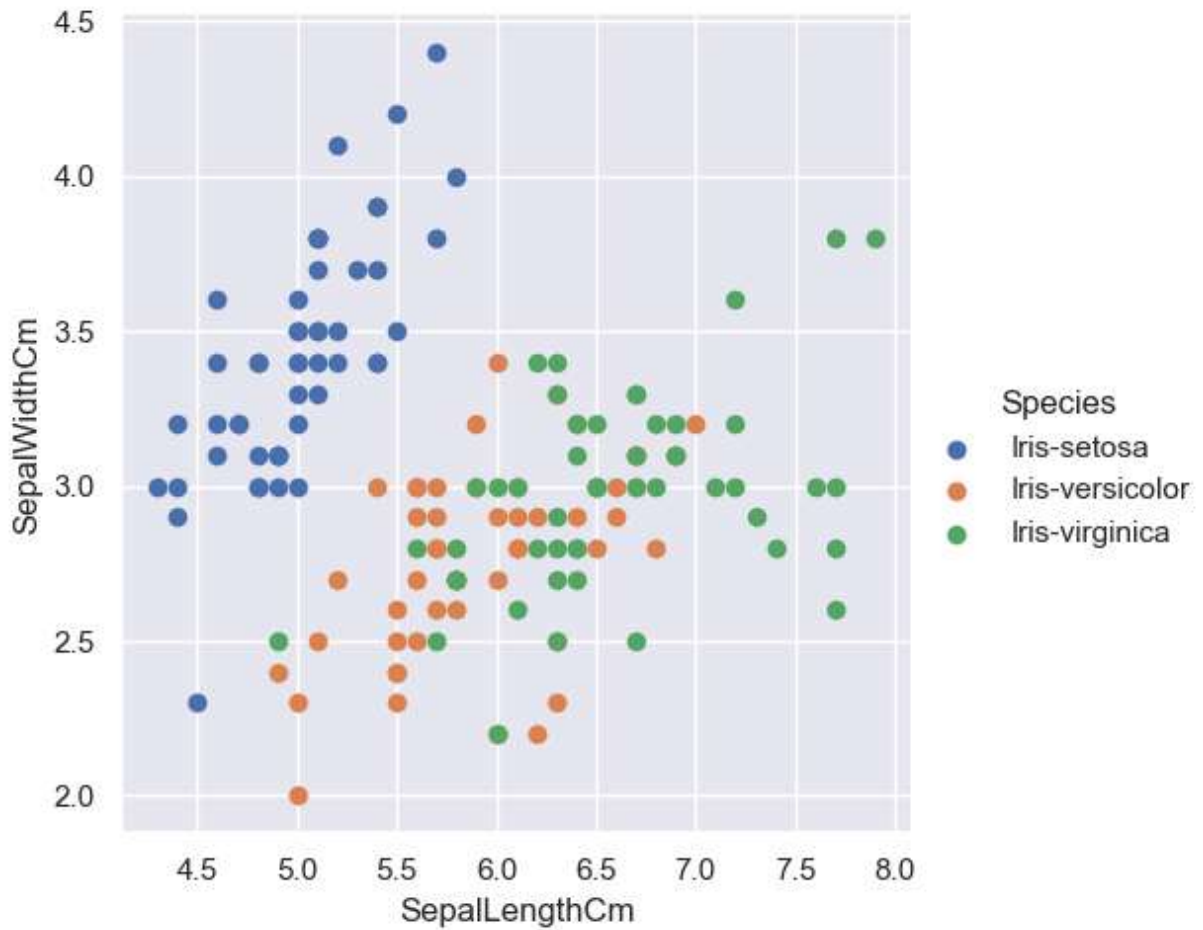
```
In [38]: fig=sns.jointplot(x='SepalLengthCm',y='SepalWidthCm',data=iris,kind="reg")
```



```
In [40]: fig=sns.jointplot(x='SepalLengthCm',y='SepalWidthCm',kind='hex',data=iris)
```



```
In [82]: import matplotlib.pyplot as plt
%matplotlib inline
sns.FacetGrid(iris,hue='Species',height=5)\
.map(plt.scatter,'SepalLengthCm','SepalWidthCm')\
.add_legend()
plt.show()
```

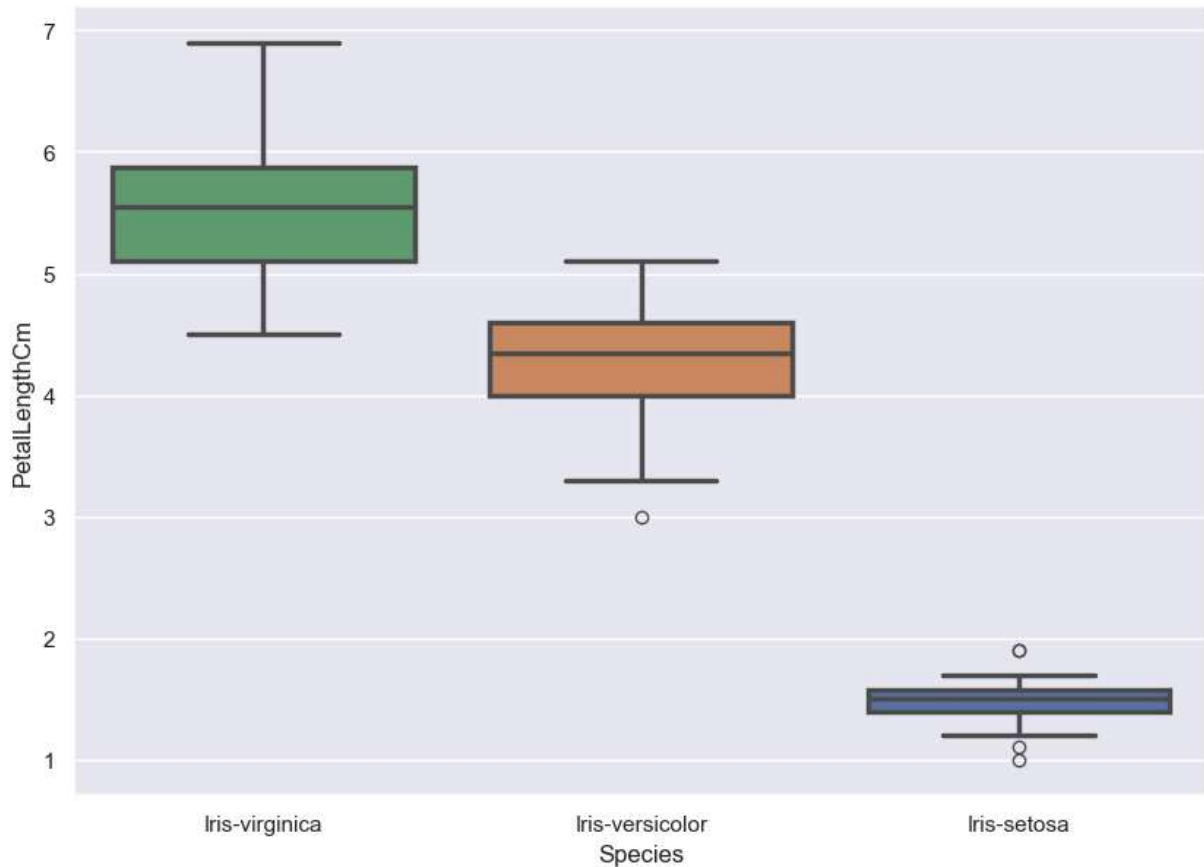


```
In [48]: iris.head()
```

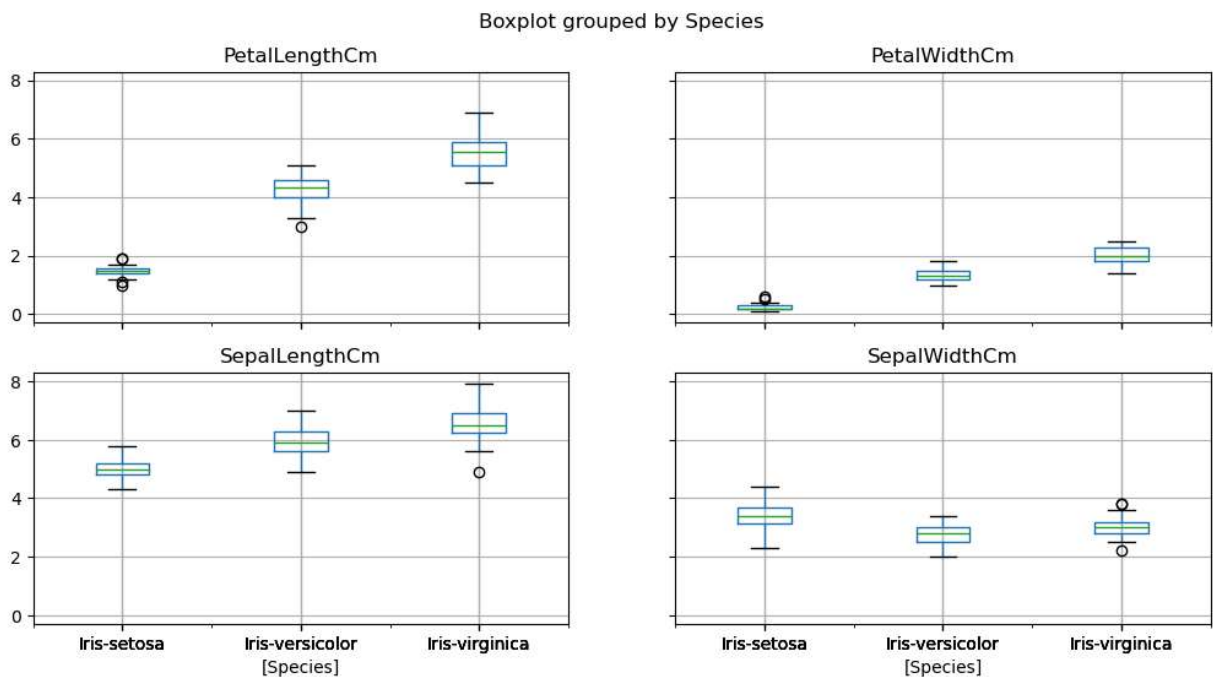
```
Out[48]:
```

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

```
In [81]: fig=plt.gcf()
fig.set_size_inches(10,7)
fig=sns.boxplot(x='Species',y='PetalLengthCm',data=iris,order=['Iris-virginica','Iris-versicolor','Iris-setosa'])
plt.show()
```

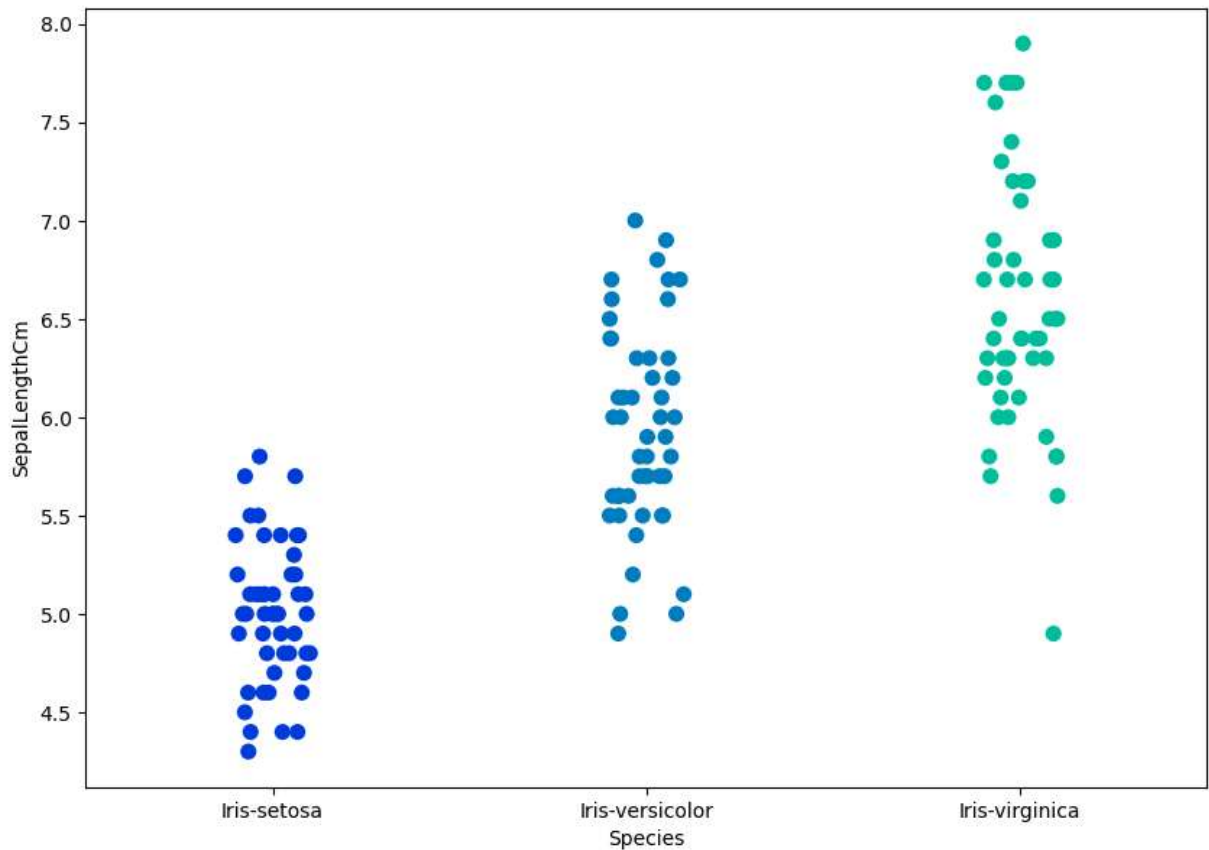


```
In [57]: #iris.drop("Id",axis=1).boxplot(by='Species',figsize=(12,6))
iris.boxplot(by='Species',figsize=(12,6))
plt.show()
```



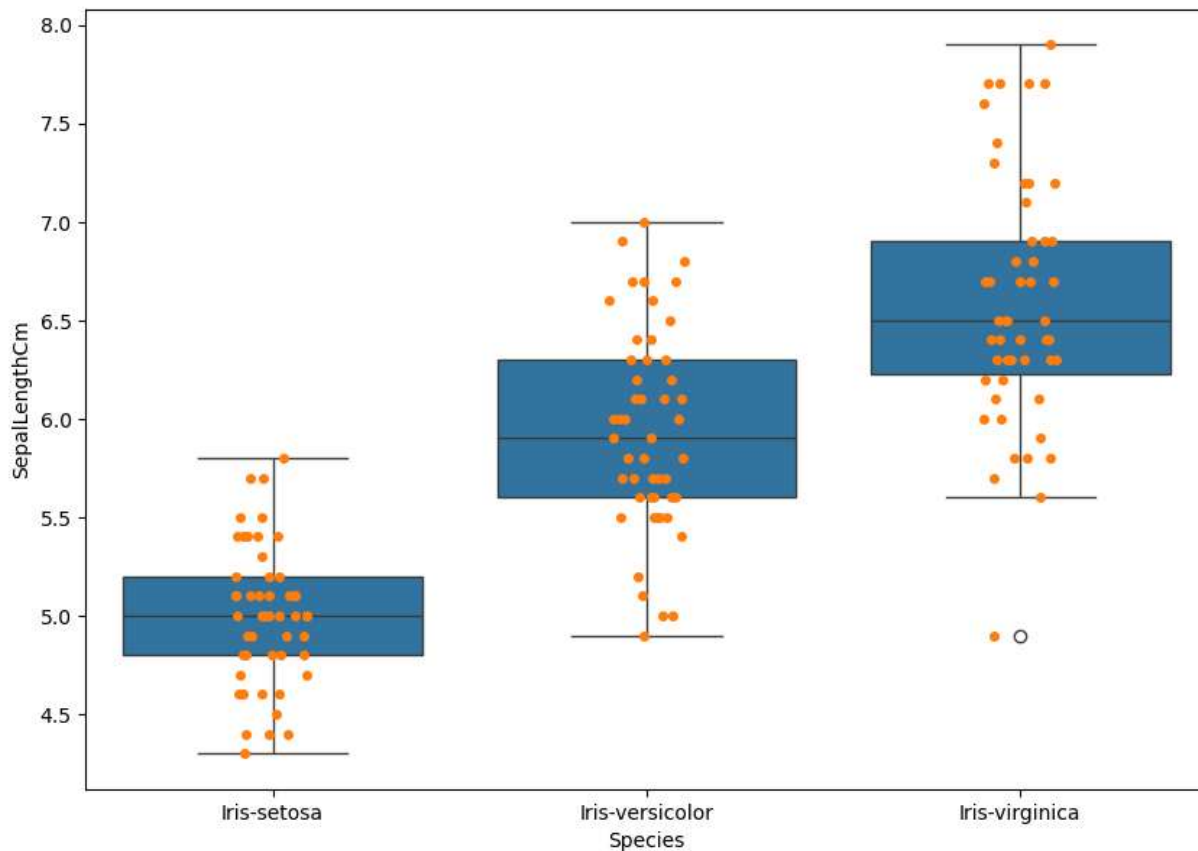
7.Strip Plot

```
In [18]: fig=plt.gcf()
fig.set_size_inches(10,7)
fig=sns.stripplot(x='Species',y='SepalLengthCm',data=iris,jitter=True,edgecolor='gr
plt.show()
```



8. Combining Box and Strip Plots

```
In [23]: fig=plt.gcf()
fig.set_size_inches(10,7)
fig=sns.boxplot(x='Species',y='SepalLengthCm',data=iris)
fig=sns.stripplot(x='Species',y='SepalLengthCm',data=iris,jitter=True,edgecolor='gr
plt.show()
```



In [11]: iris

Out[11]:

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	Iris-virginica
146	6.3	2.5	5.0	1.9	Iris-virginica
147	6.5	3.0	5.2	2.0	Iris-virginica
148	6.2	3.4	5.4	2.3	Iris-virginica
149	5.9	3.0	5.1	1.8	Iris-virginica

150 rows × 5 columns

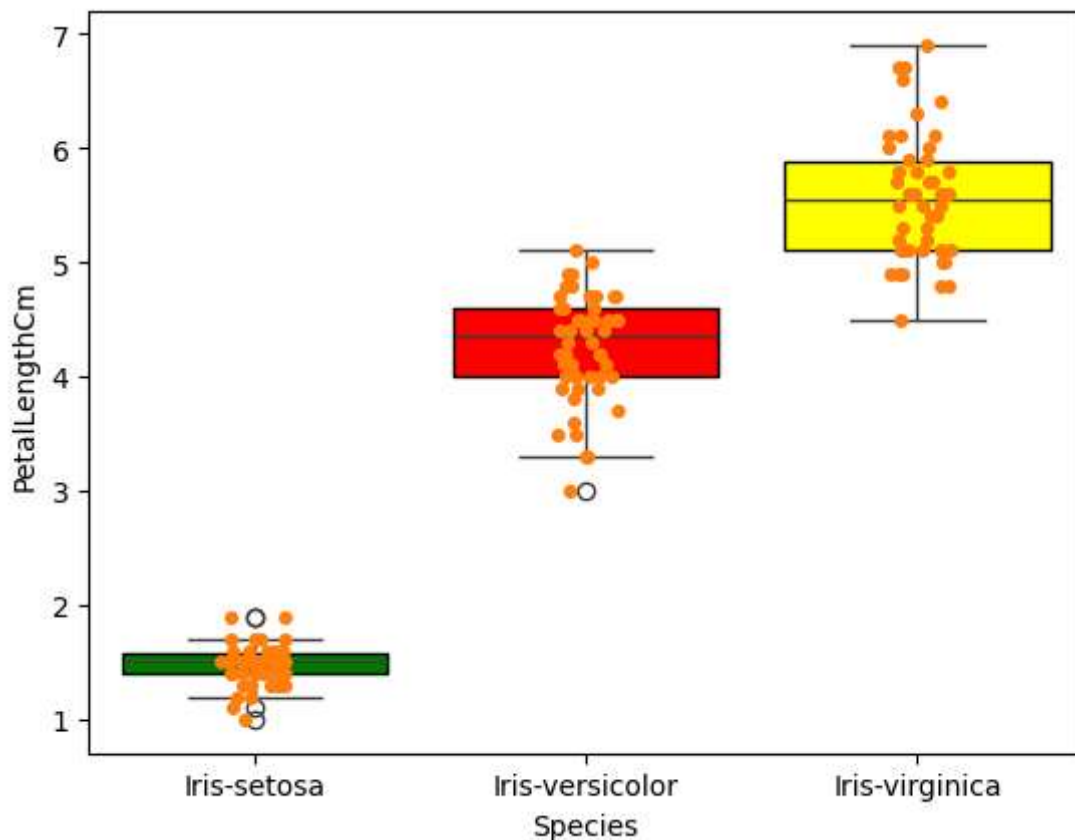
In [17]: `ax= sns.boxplot(x="Species",y="PetalLengthCm",data=iris)`
`ax= sns.stripplot(x="Species",y="PetalLengthCm",data=iris,jitter=True, edgecolor="g")`

```

boxtwo = ax.patches[2]
boxtwo.set_facecolor('yellow')
boxtwo.set_edgecolor('black')
boxthree=ax.patches[1]
boxthree.set_facecolor('red')
boxthree.set_edgecolor('black')
boxthree=ax.patches[0]
boxthree.set_facecolor('green')
boxthree.set_edgecolor('black')

plt.show()

```



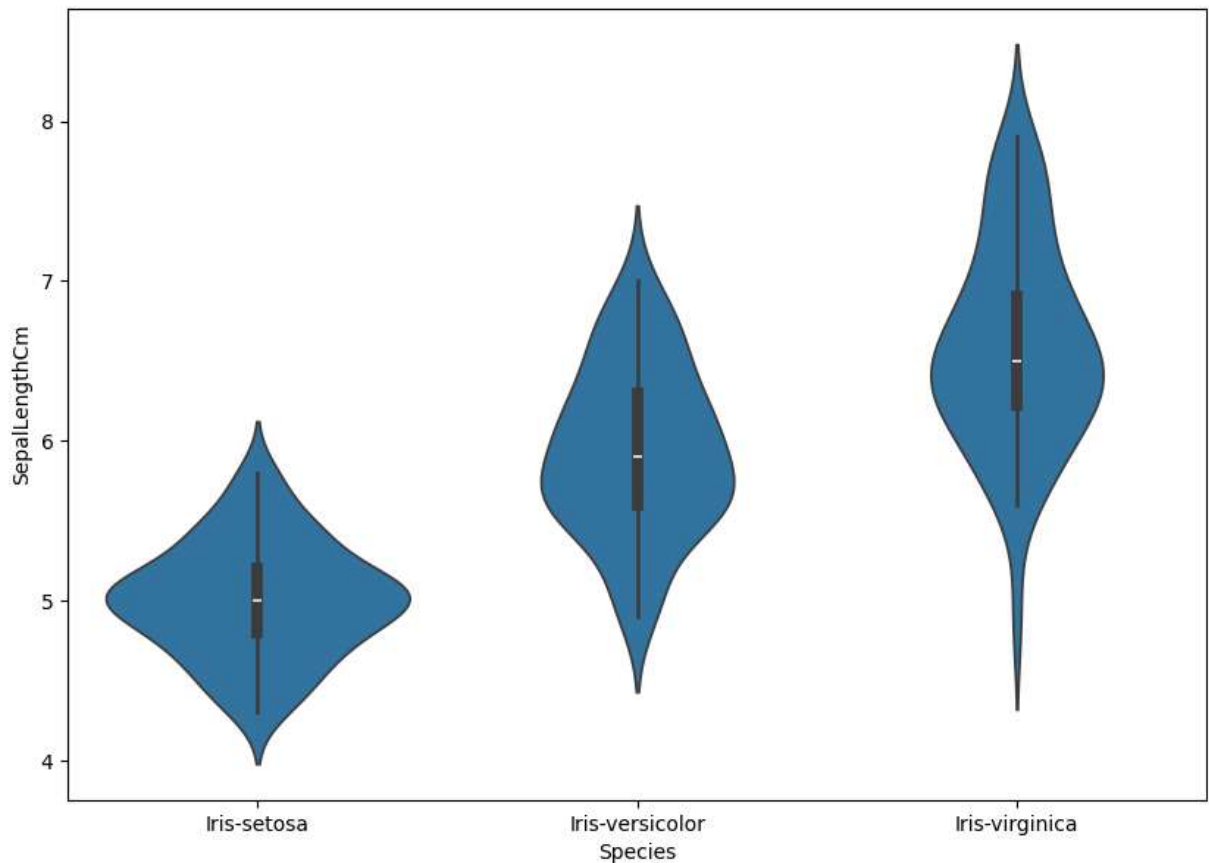
9. Violin Plot

It is used to visualize the distribution of data and its probability distribution. This chart is a combination of a Box Plot and a Density Plot that is rotated and placed on each side, to show the distribution shape of the data. The thick black bar in the centre represents the interquartile range, the thin black line extended from it represents the 95% confidence intervals, and the white dot is the median. Box Plots are limited in their display of the data, as their visual simplicity tends to hide significant details about how values in the data are distributed

```

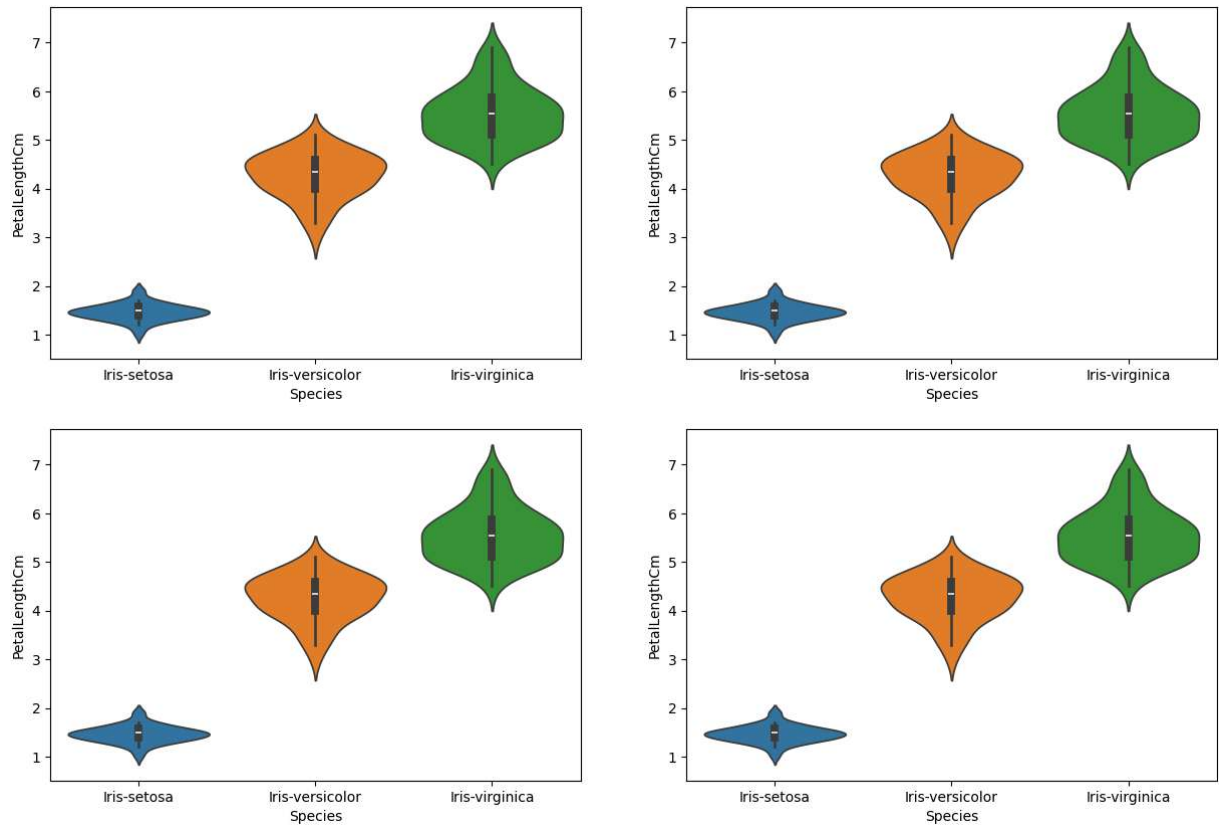
In [22]: fig=plt.gcf()
fig.set_size_inches(10,7)
fig=sns.violinplot(x='Species',y='Sepal.LengthCm',data=iris)

```



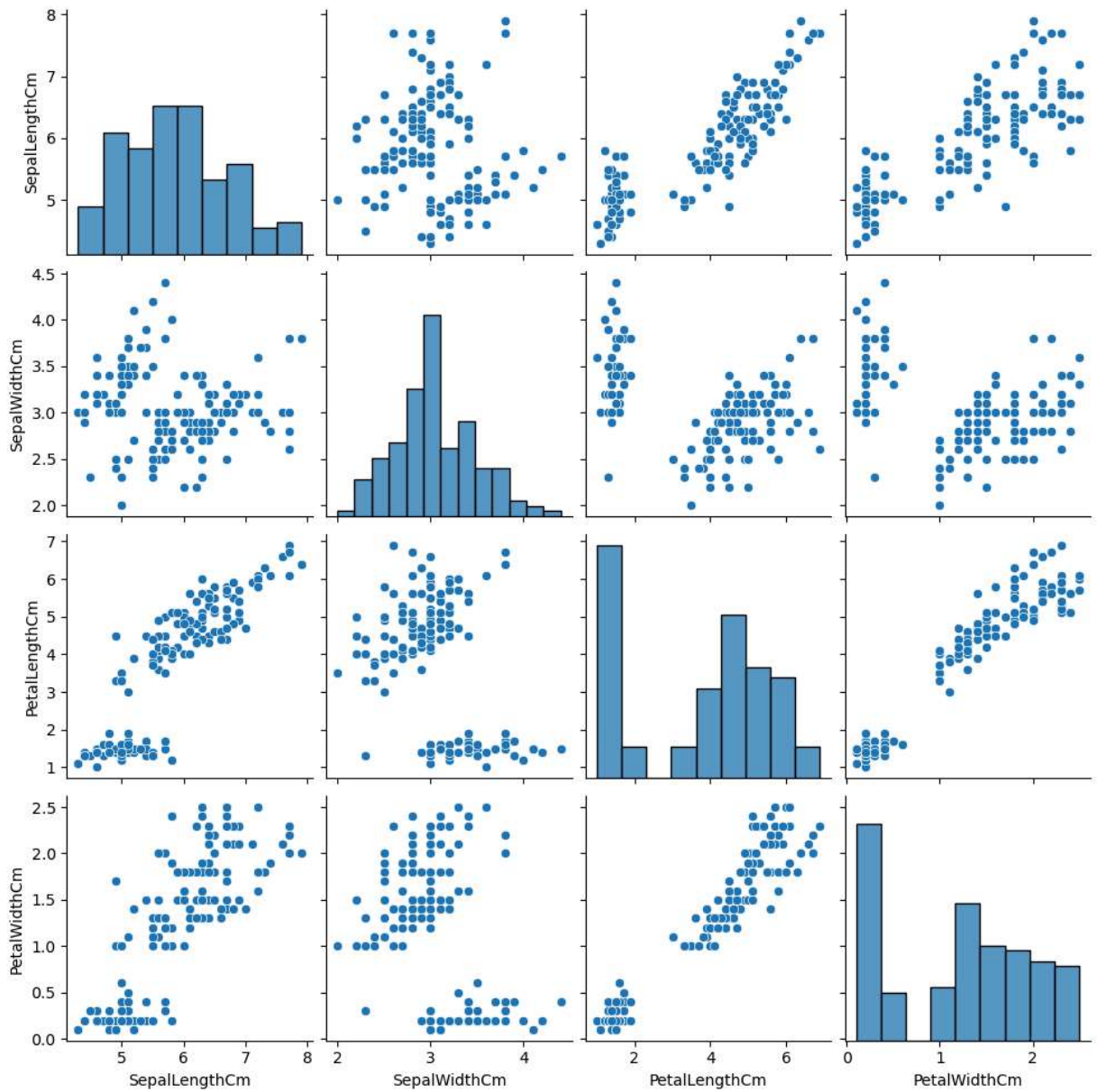
```
In [26]: plt.figure(figsize=(15,10))
plt.subplot(2,2,1)
sns.violinplot(x="Species",y="PetalLengthCm",data=iris,hue="Species")
plt.subplot(2,2,2)
sns.violinplot(x="Species",y="PetalLengthCm",data=iris,hue="Species")
plt.subplot(2,2,3)
sns.violinplot(x="Species",y="PetalLengthCm",data=iris,hue="Species")
plt.subplot(2,2,4)
sns.violinplot(x="Species",y="PetalLengthCm",data=iris,hue="Species")
```

```
Out[26]: <Axes: xlabel='Species', ylabel='PetalLengthCm'>
```



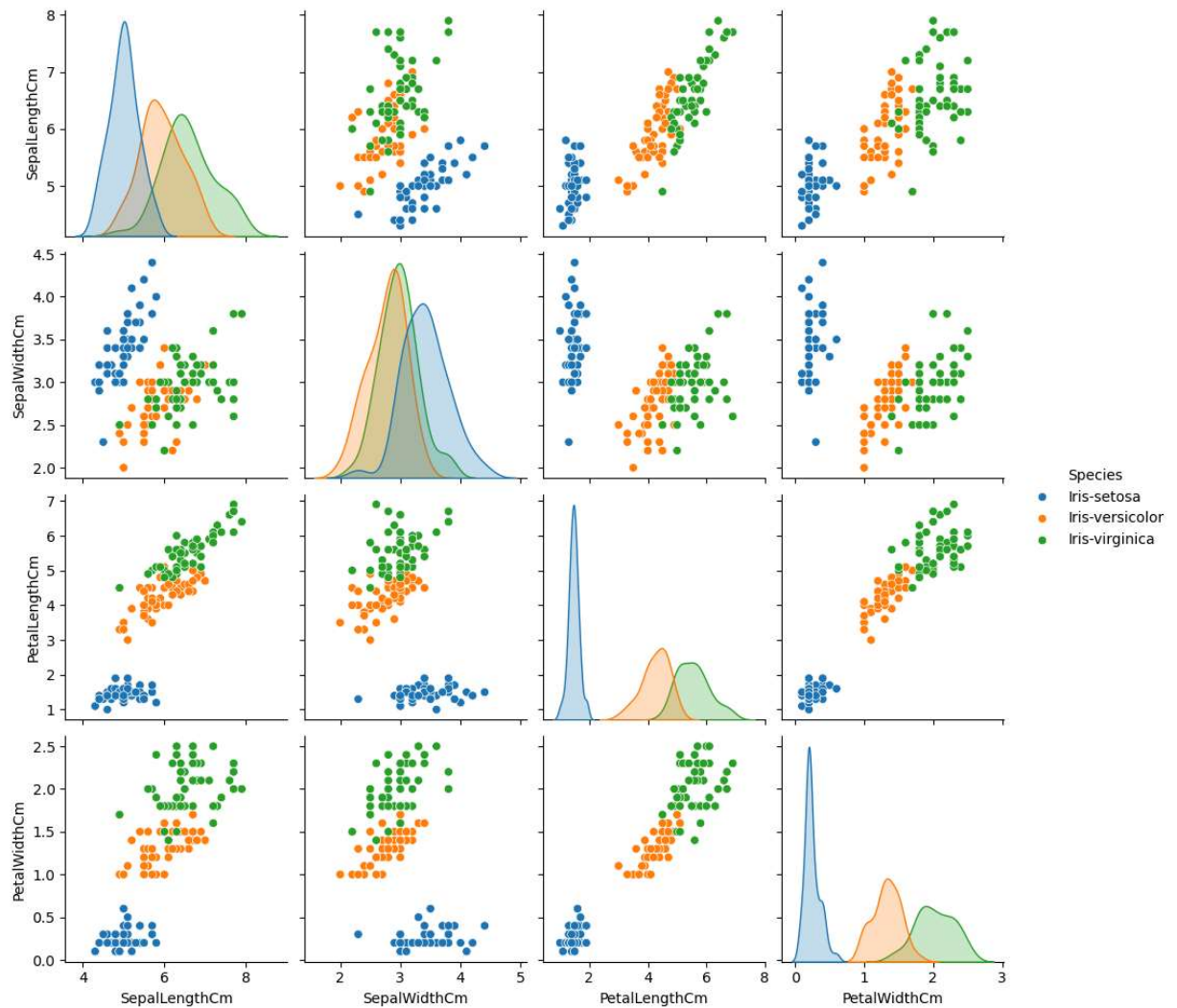
10. Pair Plot: A “pairs plot” is also known as a scatterplot, in which one variable in the same data row is matched with another variable's value, like this: Pairs plots are just elaborations on this, showing all variables paired with all the other variables.

```
In [29]: sns.pairplot(data=iris,kind='scatter',)  
plt.show()
```



```
In [30]: sns.pairplot(iris,hue='Species')
```

```
Out[30]: <seaborn.axisgrid.PairGrid at 0x12fa6c84e30>
```

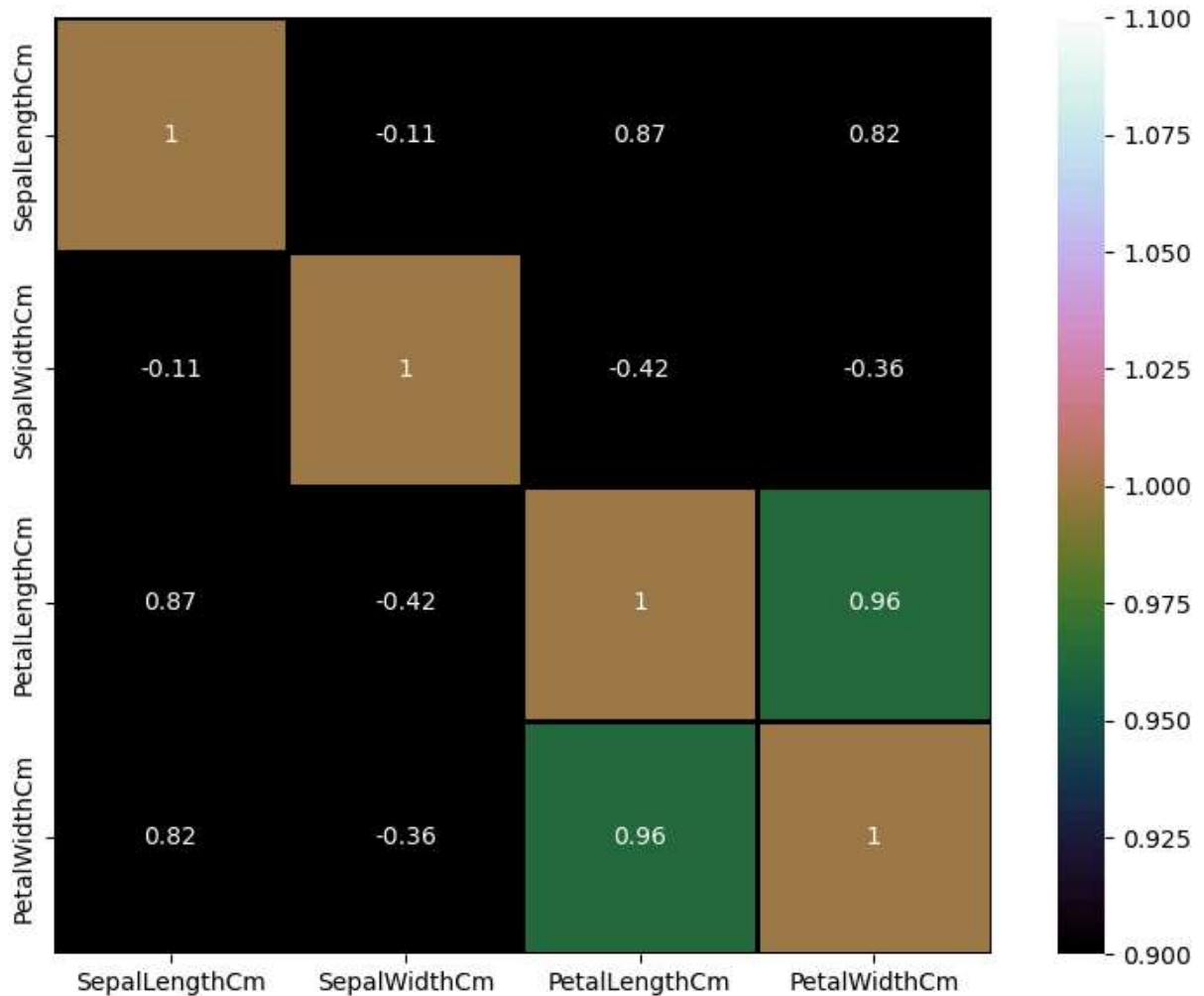


11. Heat map Heat map is used to find out the correlation between different features in the dataset. High positive or negative value shows that the features have high correlation. This helps us to select the parameters for machine learning.

```
In [35]: fig=plt.gcf()
fig.set_size_inches(10,7)
numeric_iris=iris.select_dtypes(include=['number'])
fig=sns.heatmap(numeric_iris.corr(),

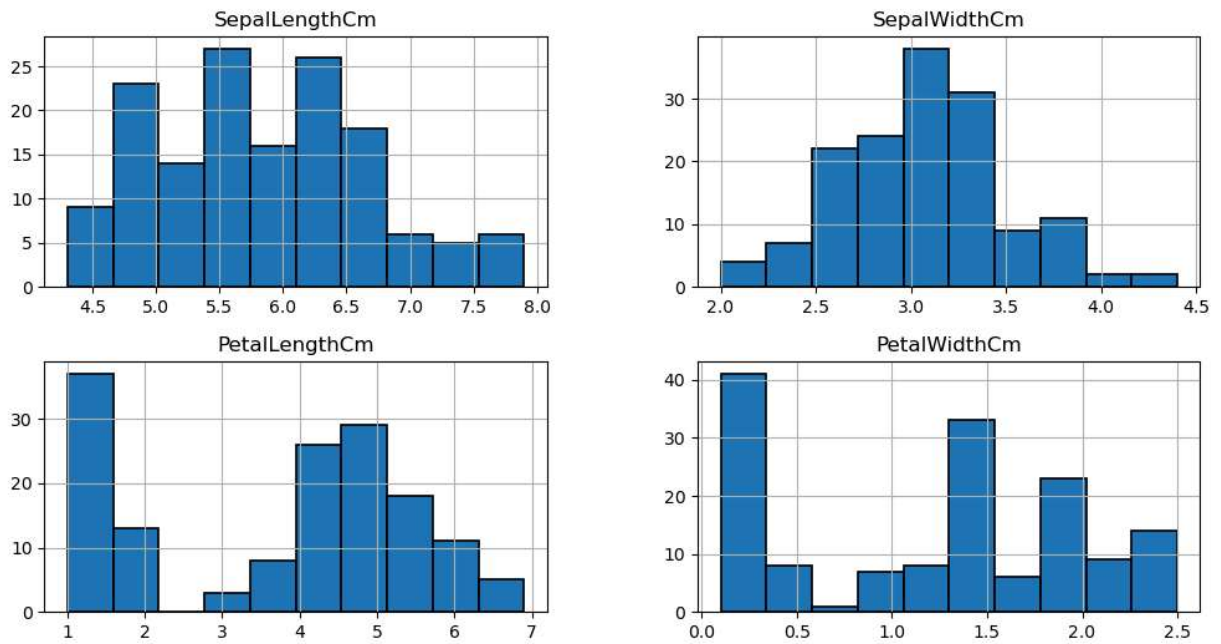
                annot=True,
                cmap='cubehelix',
                linewidths=1,
                linecolor='k',
                square=True,
                mask=False,
                vmin=1,
                vmax=1,
                cbar_kws={"orientation":"vertical"},
                cbar=True)

plt.show()
```



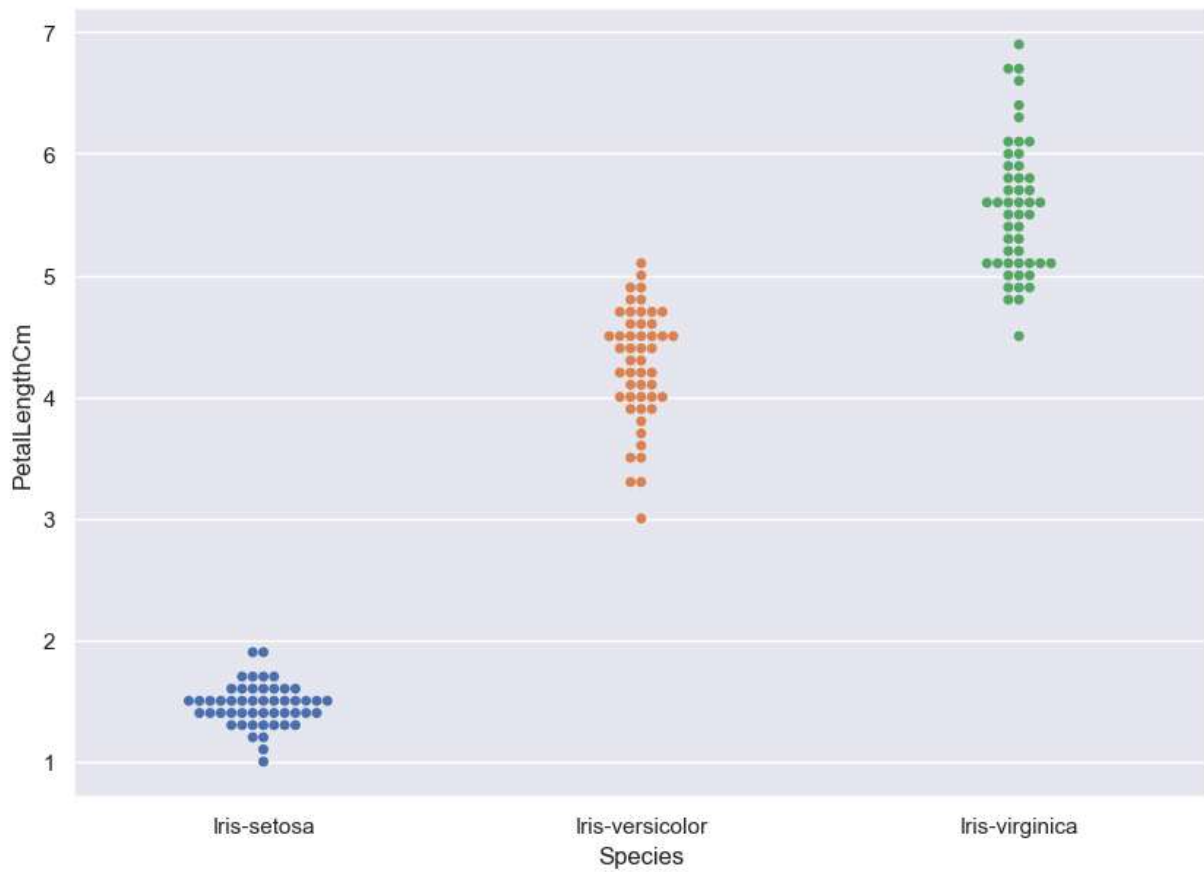
12. Distribution plot: The distribution plot is suitable for comparing range and distribution for groups of numerical data. Data is plotted as value points along an axis. You can choose to display only the value points to see the distribution of values, a bounding box to see the range of values, or a combination of both as shown here. The distribution plot is not relevant for detailed analysis of the data as it deals with a summary of the data distribution.

```
In [38]: iris.hist(edgecolor='black',linewidth=1.2)
fig=plt.gcf()
fig.set_size_inches(12,6)
```

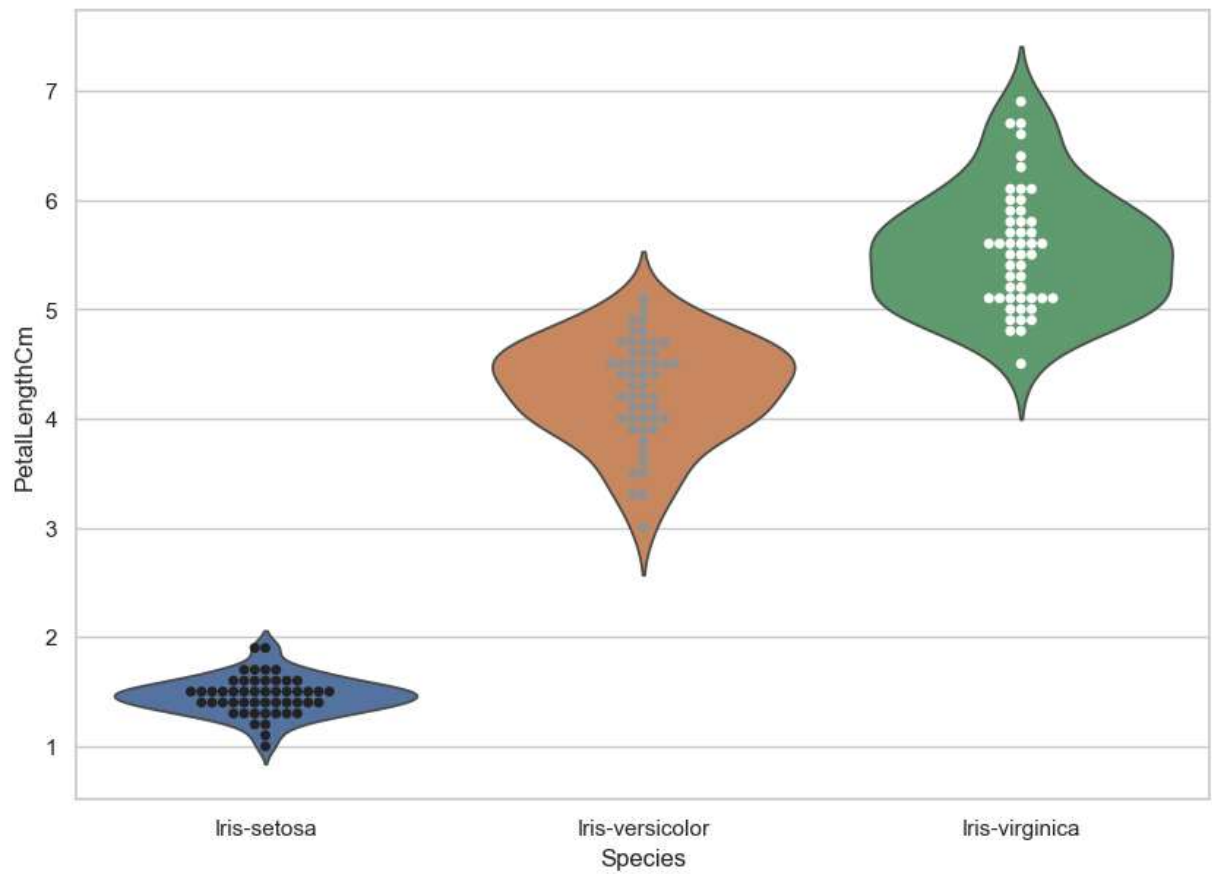


13. Swarm plot It looks a bit like a friendly swarm of bees buzzing about their hive. More importantly, each data point is clearly visible and no data are obscured by overplotting. A beeswarm plot improves upon the random jittering approach to move data points the minimum distance away from one another to avoid overlays. The result is a plot where you can see each distinct data point, like shown in below plot

```
In [43]: sns.set(style="darkgrid")
fig=plt.gcf()
fig.set_size_inches(10,7)
fig = sns.swarmplot(x="Species", y="PetalLengthCm",data=iris,hue="Species")
plt.show()
```

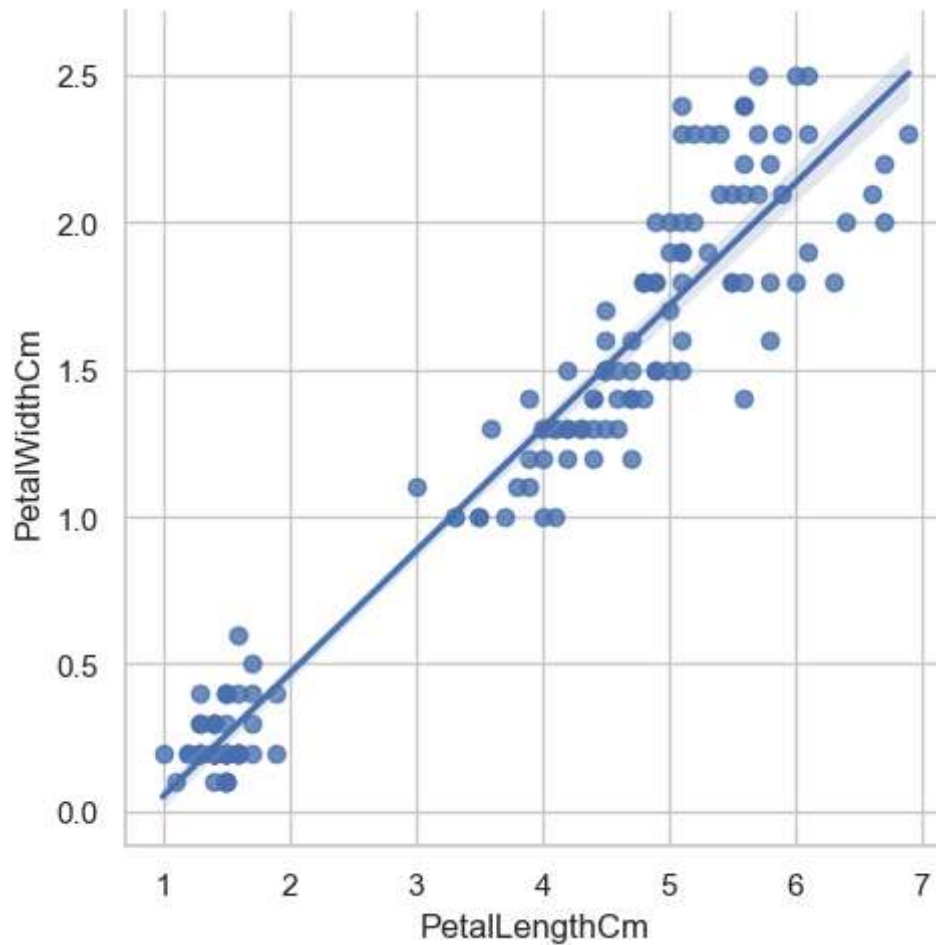


```
In [47]: sns.set(style="whitegrid")
fig=plt.gcf()
fig.set_size_inches(10,7)
ax = sns.violinplot(x = "Species", y="PetalLengthCm", data=iris,inner=None,hue="Spe
ax = sns.swarmplot(x= "Species", y = "PetalLengthCm",data=iris,color="white",edgeco
```



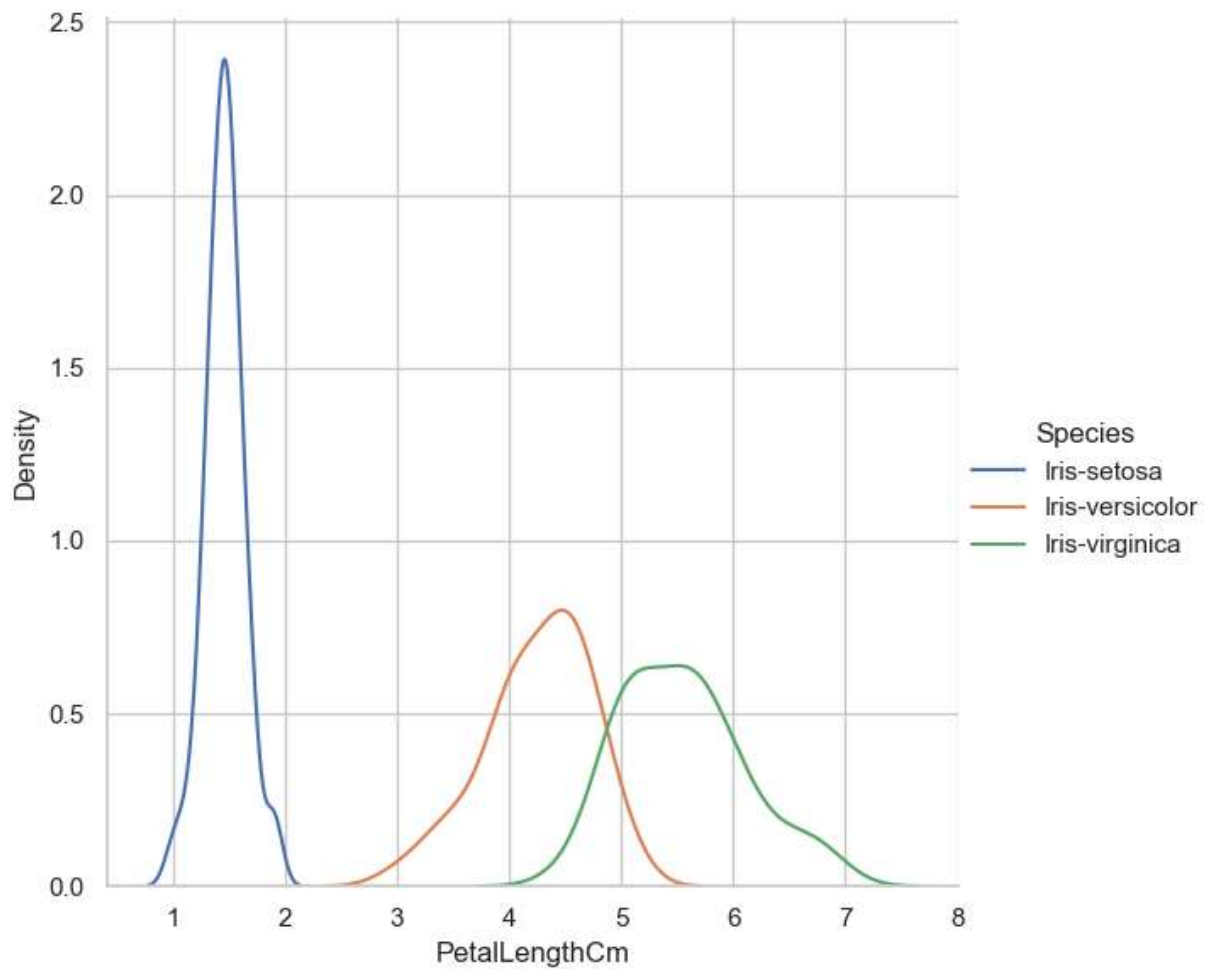
17. LM Plot

```
In [51]: fig=sns.lmplot(x="PetalLengthCm",y="PetalWidthCm",data=iris,)
```



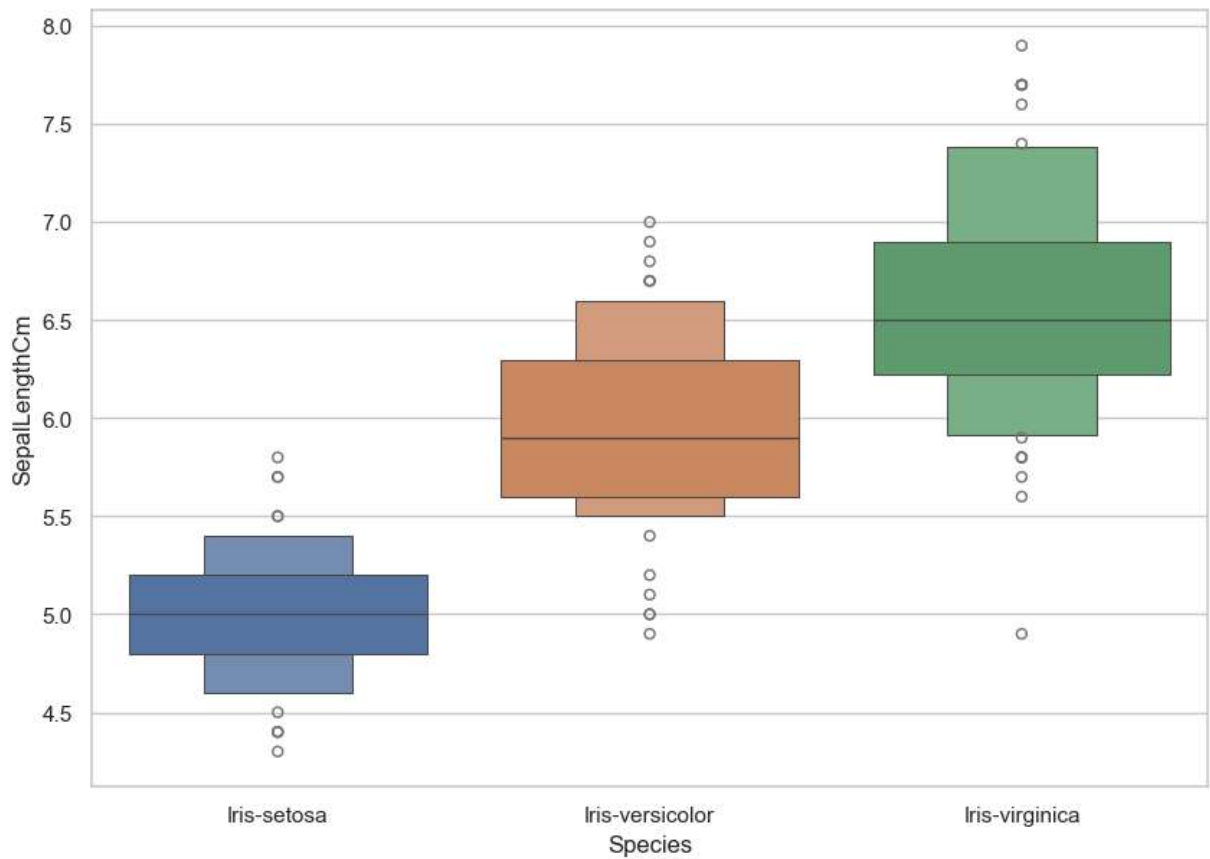
18. FacetGrid

```
In [57]: sns.FacetGrid(data=iris, hue="Species", height=6)\
        .map(sns.kdeplot, "PetalLengthCm")\
        .add_legend()
plt.ioff()
plt.show()
```



BoXen Plot

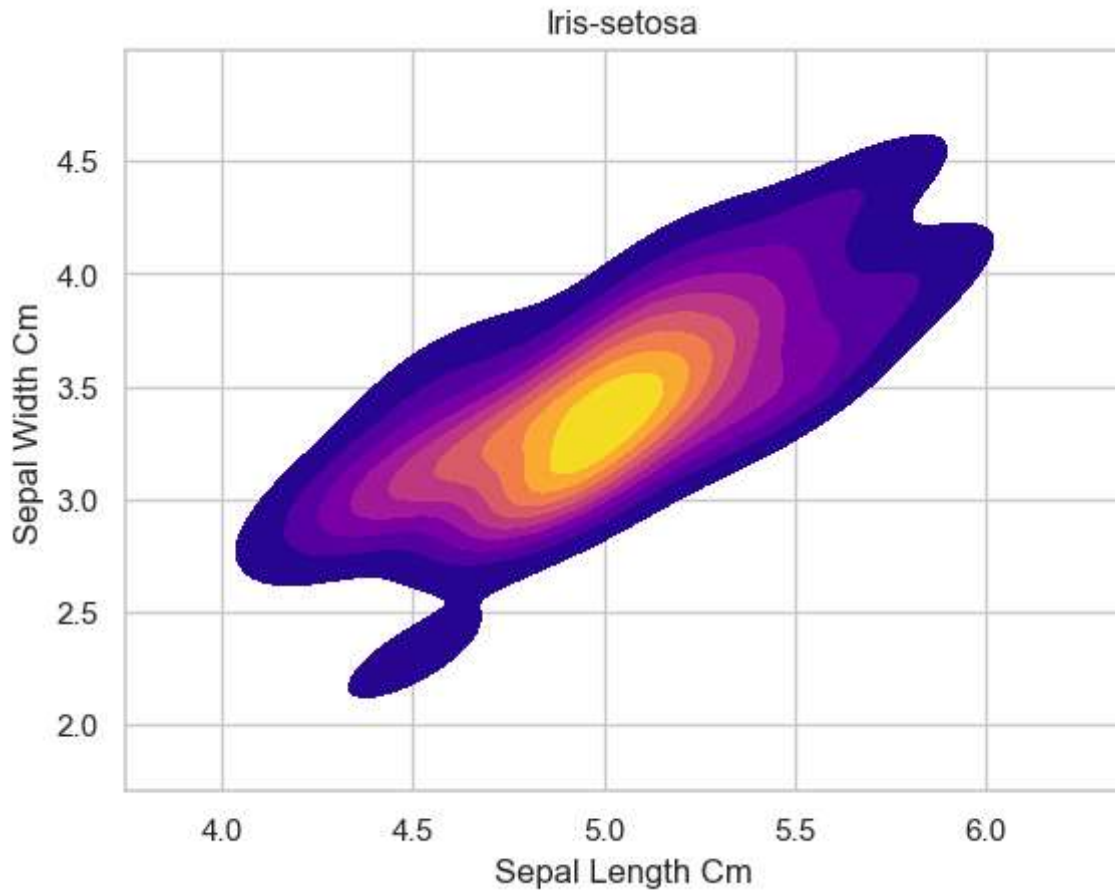
```
In [61]: fig=plt.gcf()
fig.set_size_inches(10,7)
fig=sns.boxenplot(x='Species',y='SepalLengthCm',data=iris,hue='Species')
plt.show()
```



KDE Plot

```
In [65]: sub=iris[iris['Species']=='Iris-setosa']
sns.kdeplot(data=sub,x='SepalLengthCm',y='SepalWidthCm',cmap="plasma", shade=True,
plt.title('Iris-setosa')
plt.xlabel('Sepal Length Cm')
plt.ylabel('Sepal Width Cm')

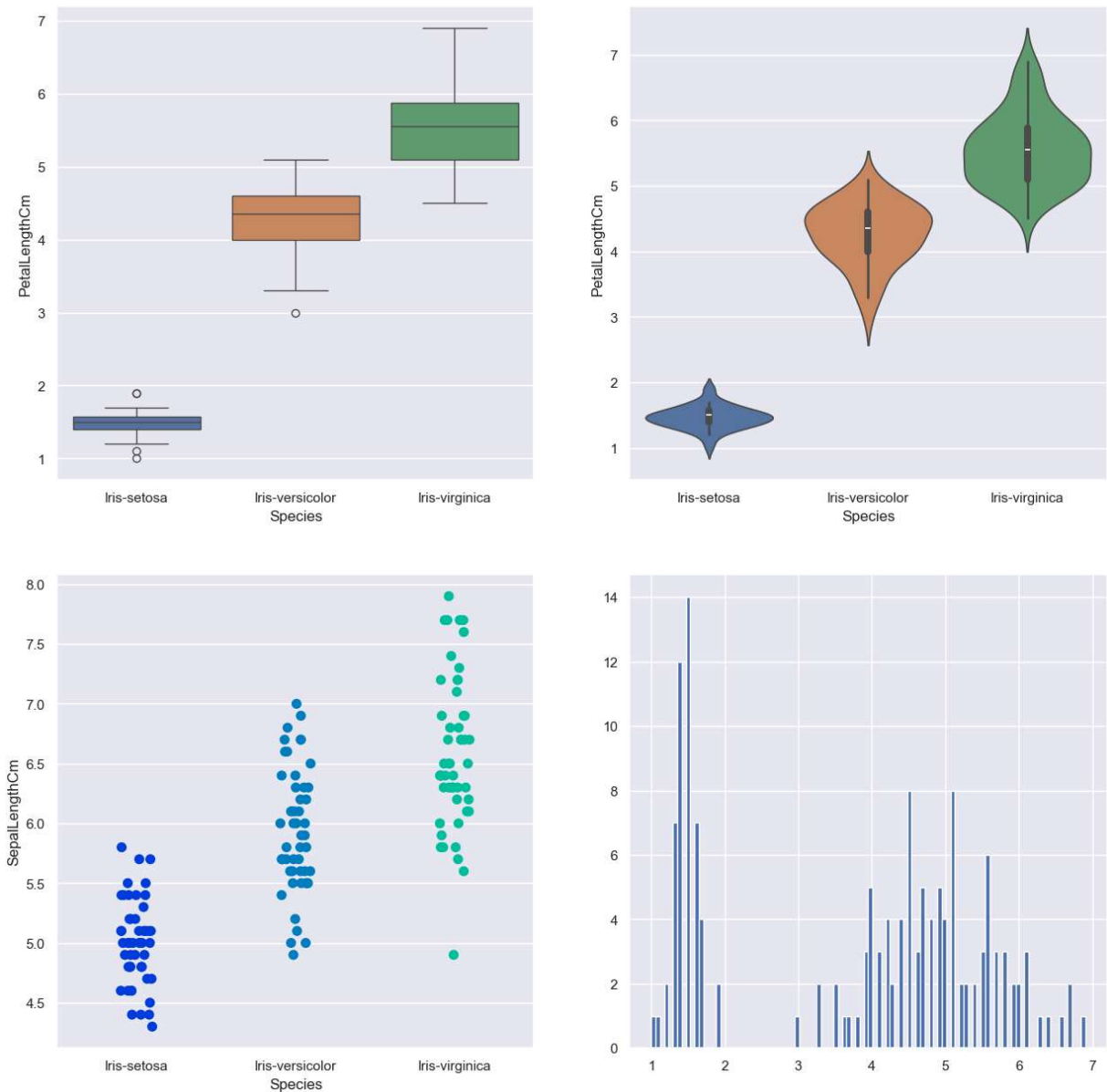
plt.show()
```



Dashboard

```
In [70]: sns.set_style('darkgrid')
f, axes = plt.subplots(2, 2, figsize=(15, 15))

k1 = sns.boxplot(x="Species", y="PetalLengthCm", data=iris, ax=axes[0, 0], hue="Species")
k2 = sns.violinplot(x="Species", y="PetalLengthCm", data=iris, ax=axes[0, 1], hue="Species")
k3 = sns.stripplot(x="Species", y="SepalLengthCm", data=iris, jitter=True, edgecolor='gray')
# axes[1, 1].hist(iris.hist, bin=10)
axes[1, 1].hist(iris.PetalLengthCm, bins=100)
# k2.set(xlim=(-1, 0.8))
plt.show()
```

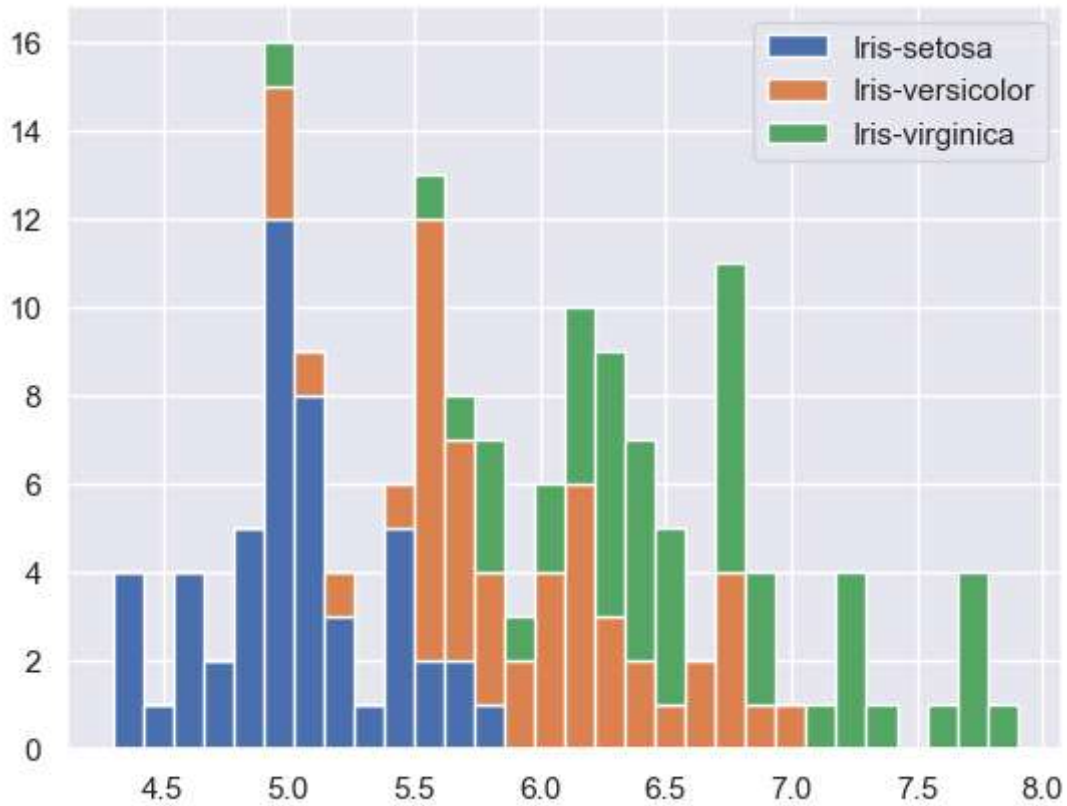


Stacked Histogram

```
In [72]: iris['Species'] = iris['Species'].astype('category')
         #iris.head
```

```
In [74]: list1=list()
         mylabels=list()
         for gen in iris.Species.cat.categories:
             list1.append(iris[iris.Species==gen].Sepal.LengthCm)
             mylabels.append(gen)

         h=plt.hist(list1,bins=30,stacked=True,rwidth=1,label=mylabels)
         plt.legend()
         plt.show()
```



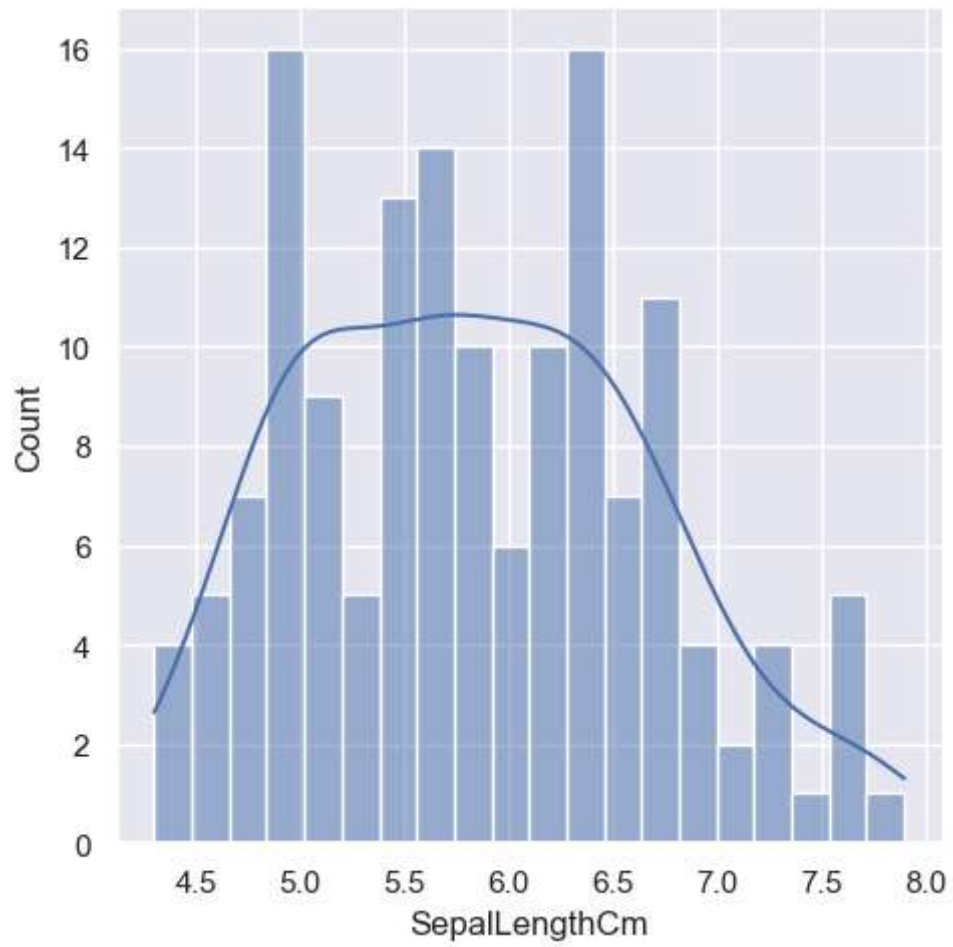
Area plot

```
In [77]: iris.plot.area(y=['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm'], alpha=0.4, figsize  
plt.show())
```



Distplot

```
In [80]: sns.displot(iris['SepalLengthCm'], kde=True, bins=20)  
plt.show()
```



```
In [ ]:
```